Projet Secure Access Manager

PROJET D'EXAMEN :

Dans le cadre du module de programmation en langage C, vous devez réaliser un mini système de gestion des accès sécurisés. Ce projet s'inscrit dans le domaine de la cybersécurité et vise à simuler la gestion d'utilisateurs dans un système informatique.

Auteurs groupe 7 :

- Cheikh Magueye Thiam
- Mouhamadou Mansour Niang
- Diarra Mané

Table des matières

Objectif Général : 2
1.1-Menu principal du projet Secure Access Manager : code du fichier main.c 3
1.2- gestion des utilisateurs (ajout / affichage / suppression) : code du fichier user.c et header user.h 4
1.3-allocation dynamique (malloc / realloc / free) : code du fichier memoir.c 7
1.4-Sauvegarde et chargement : code du fichier fichiers.c 9
1.5-règles de sécurité via pointeurs de fonctions :..... 10
Code du fichier securite.c..... 10
Conclusion :..... 13

Secure Acess

Objectif Général :

Développer une application en langage C permettant de : - gérer des utilisateurs, - stocker leurs informations, - analyser leur accès, - appliquer des règles simples de sécurité.

Secure Access

1.1-Menu principal du projet Secure Access Manager : code du fichier main.c

```c
#include <stdio.h>
#include "user.h"

static void afficher_menu(void)
{
    printf("\n");
    printf("SecureAccess Manager\n");
    printf("\n");
    printf(" 1. Ajouter un utilisateur       \n");
    printf(" 2. Afficher les utilisateurs    \n");
    printf(" 3. Sauvegarder                  \n");
    printf(" 4. Charger                      \n");
    printf(" 5. Verifier la securite         \n");
    printf(" 6. Supprimer un utilisateur     \n");
    printf(" 0. Quitter                      \n");
    printf("\n");
    printf("  Votre choix : ");
}

```

Secure

```c
int main(void)
{
    Gestionnaire g;
    init_gestionnaire(&g);

    int choix;
    do
    {
        afficher_menu();
        scanf(" %d", &choix);

        switch (choix)
        {
        case 1:
            ajouter_utilisateur(&g);
            break;
        case 2:
            afficher_utilisateurs(&g);
            break;
        case 3:
            sauvegarder(&g);
            break;
        case 4:
            charger(&g);
            break;
        case 5:
            verifier_securite_tous(&g);
            break;
        case 6:
            supprimer_utilisateur(&g);
            break;
        case 0:
            printf("  Au revoir !\n");
            break;
        default:
            printf("  Choix invalide.\n");
        }
    } while (choix != 0);

    liberer_gestionnaire(&g);
    return 0;
}
```

1.2- gestion des utilisateurs (ajout / affichage / suppression) : code du fichier user.c et header user.h

```c
#include <stdio.h>
#include <string.h>
#include "user.h"

void ajouter_utilisateur(Gestionnaire *g) {
    if (!agrandir_si_necessaire(g)) return;

    User *u = &g->liste[g->nb];
    u->id = g->prochain_id++;

    printf("  Nom d'utilisateur : ");
    scanf(" %49s", u->nom);
    printf("  Mot de passe      : ");
    scanf(" %49s", u->motdepasse);
    printf("  Niveau d'acces (1=invite, 2=user, 3=admin) : ");
    scanf(" %d", &u->niveau_acces);

    g->nb++;
    printf("  Utilisateur ajoute avec ID %d.\n", u->id);
}

void afficher_utilisateurs(const Gestionnaire *g) {
    if (g->nb == 0) {
        printf("  Aucun utilisateur enregistre.\n");
        return;
    }
    printf("\n  %-5s %-20s %-20s %s\n", "ID", "Nom", "Mot de passe", "Niveau");
    printf("  %s\n", "--------------------------------------------------");
    for (int i = 0; i < g->nb; i++) {
        const User *u = &g->liste[i];
        printf("  %-5d %-20s %-20s %d\n",
                u->id, u->nom, u->motdepasse, u->niveau_acces);
    }
}
```

```c
void supprimer_utilisateur(Gestionnaire *g) {
    if (g->nb == 0) {
        printf("  Aucun utilisateur a supprimer.\n");
        return;
    }
    int id;
    printf("  ID de l'utilisateur a supprimer : ");
    scanf(" %d", &id);

    for (int i = 0; i < g->nb; i++) {
        if (g->liste[i].id == id) {
            for (int j = i; j < g->nb - 1; j++)
                g->liste[j] = g->liste[j + 1];
            g->nb--;
            printf("  Utilisateur ID %d supprime.\n", id);
            return;
        }
    }
    printf("  Utilisateur ID %d introuvable.\n", id);
}

```

```c
#ifndef USER_H
#define USER_H

#define MAX_NOM     50
#define MAX_MDP     50
#define FICHIER_DAT "users.dat"

/* — Structure utilisateur — */
typedef struct {
    int  id;
    char nom[MAX_NOM];
    char motdepasse[MAX_MDP];
    int  niveau_acces;   /* 1=invité  2=user  3=admin */
} User;

/* — Gestionnaire (tableau dynamique) — */
typedef struct {
    User *liste;
    int   nb;
    int   capacite;
    int   prochain_id;
} Gestionnaire;
```

Secu

```c
/* memoire.c */
void init_gestionnaire(Gestionnaire *g);
void liberer_gestionnaire(Gestionnaire *g);
int  agrandir_si_necessaire(Gestionnaire *g);

/* user.c */
void ajouter_utilisateur(Gestionnaire *g);
void afficher_utilisateurs(const Gestionnaire *g);
void supprimer_utilisateur(Gestionnaire *g);

/* fichiers.c */
void sauvegarder(const Gestionnaire *g);
void charger(Gestionnaire *g);

/* securite.c */
void verifier_securite_tous(const Gestionnaire *g);

#endif
```

1.3-allocation dynamique (malloc / realloc / free) : code du fichier memoir.c

```c
#include <stdio.h>
#include <stdlib.h>
#include "user.h"

void init_gestionnaire(Gestionnaire *g) {
    g->liste       = NULL;
    g->nb          = 0;
    g->capacite    = 0;
    g->prochain_id = 1;
}

void liberer_gestionnaire(Gestionnaire *g) {
    free(g->liste);
    g->liste    = NULL;
    g->nb       = 0;
    g->capacite = 0;
}

int agrandir_si_necessaire(Gestionnaire *g) {
    if (g->nb >= g->capacite) {
        int nouvelle_cap = (g->capacite == 0) ? 4 : g->capacite * 2;
        User *tmp = realloc(g->liste, nouvelle_cap * sizeof(User));
        if (!tmp) {
            fprintf(stderr, "Erreur : allocation memoire echouee.\n");
            return 0;
        }
        g->liste    = tmp;
        g->capacite = nouvelle_cap;
    }
    return 1;
}
```

Secure

1.4-Sauvegarde et chargement : code du fichier fichiers.c

```c
#include <stdio.h>
#include <stdlib.h>
#include "user.h"

void sauvegarder(const Gestionnaire *g)
{
    FILE *f = fopen(FICHIER_DAT, "wb");
    if (!f)
    {
        fprintf(stderr, "  Erreur : impossible d'ouvrir %s.\n", FICHIER_DAT);
        return;
    }
    fwrite(&g->nb, sizeof(int), 1, f);
    fwrite(&g->prochain_id, sizeof(int), 1, f);
    fwrite(g->liste, sizeof(User), g->nb, f);
    fclose(f);
    printf("  %d utilisateur(s) sauvegardes dans %s.\n", g->nb, FICHIER_DAT);
}

void charger(Gestionnaire *g)
{
    FILE *f = fopen(FICHIER_DAT, "rb");
    if (!f)
    {
        printf("  Aucun fichier %s trouve.\n", FICHIER_DAT);
        return;
    }
```

Secu

```c

    int nb = 0, prochain_id = 1;
    fread(&nb, sizeof(int), 1, f);
    fread(&prochain_id, sizeof(int), 1, f);

    liberer_gestionnaire(g);
    g->liste = malloc(nb * sizeof(User));
    if (!g->liste)
    {
        fprintf(stderr, "  Erreur memoire lors du chargement.\n");
        fclose(f);
        return;
    }

    fread(g->liste, sizeof(User), nb, f);
    fclose(f);

    g->nb = nb;
    g->capacite = nb;
    g->prochain_id = prochain_id;
    printf("  %d utilisateur(s) charges depuis %s.\n", nb, FICHIER_DAT);
}

```

1.5-règles de sécurité via pointeurs de fonctions :

Code du fichier securite.c

Secure A

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include "user.h"

/* — Règles de sécurité — */

static int regle_mdp_court(const User *u) {
    return (int)strlen(u->motdepasse) >= 6;
}

static int regle_mdp_chiffre(const User *u) {
    for (int i = 0; u->motdepasse[i]; i++)
        if (isdigit((unsigned char)u->motdepasse[i])) return 1;
    return 0;
}

static int regle_niveau_valide(const User *u) {
    return (u->niveau_acces >= 1 && u->niveau_acces <= 3);
}

static int regle_nom_non_vide(const User *u) {
    return (strlen(u->nom) > 0);
}
```

Secu

```c
typedef int (*RegleSecurite)(const User *);

static RegleSecurite regles[] = {
    regle_mdp_court,
    regle_mdp_chiffre,
    regle_niveau_valide,
    regle_nom_non_vide
};

static const char *descriptions[] = {
    "Mot de passe >= 6 caracteres",
    "Mot de passe contient un chiffre",
    "Niveau d'acces valide (1-3)",
    "Nom non vide"
};

#define NB_REGLES (int)(sizeof(regles) / sizeof(regles[0]))

/* — Rapport pour un utilisateur — */
static void verifier_securite_user(const User *u) {
    int score = 0;
    printf("\n  --- Rapport : %s (ID %d) ---\n", u->nom, u->id);
    for (int i = 0; i < NB_REGLES; i++) {
        int ok = regles[i](u);   /* appel via pointeur de fonction */
        printf("  [%s] %s\n", ok ? "OK" : "!!", descriptions[i]);
        if (ok) score++;
    }
    printf("  Score : %d/%d — %s\n",
           score, NB_REGLES,
           score == NB_REGLES ? "Compte SECURISE" : "Compte A RISQUE");
}
```

```c
/* — Vérification de tous les utilisateurs — */
void verifier_securite_tous(const Gestionnaire *g) {
    if (g->nb == 0) {
        printf("  Aucun utilisateur a verifier.\n");
        return;
    }
    for (int i = 0; i < g->nb; i++)
        verifier_securite_user(&g->liste[i]);
}

```

Conclusion :

En conclusion, ce projet nous a permis de concevoir un mini système de gestion des accès sécurisés en langage C. Il nous a aidé à mettre en pratique plusieurs notions importantes comme l'utilisation des structures, la gestion des fichiers, les pointeurs et l'allocation dynamique de mémoire. Grâce à ce projet, nous avons également mieux compris certains principes de base de la cybersécurité, notamment la gestion des utilisateurs et la vérification des règles de sécurité. Cette expérience constitue donc une bonne introduction au développement de systèmes sécurisés.

Secure Access